

Trust the Model Where It Trusts Itself

MACURA: the paper, and my project on a racing drone

Silver Gjeka

Department of Computer Science
Master's Degree in Artificial Intelligence
University of Verona

October 4, 2026

Paper: B. Frauenknecht, A. Eisele, D. Subhasish, F. Solowjow, S. Trimpe (RWTH Aachen), ICML 2024,
arXiv:2405.19014

Project code: github.com/silvergjeka22/macura-drone

Outline

Part I – The paper

1. The problem: when and where to trust a model
2. Background: SAC, ensembles, Dyna, MBPO and M2AC
3. MACURA: stop imagining where the model is unsure
4. The four algorithms side by side
5. Noise, results and what matters

Part II – My project

6. The task: a racing drone with wind and a chute
7. Implementation and a fair protocol
8. Results: learning, testing, trust, one flight
9. Limitations and conclusion

Part I

The Paper

How MACURA decides *where* to trust its learned model

The Problem: When and Where to Trust a Model

- Deep RL needs **millions of interactions**. On a real robot or drone, each one is slow, expensive or risky.
- **Model-based RL** learns a model of the world and uses it to **imagine** extra experience.
- But a learned model is never perfect, and training on its **wrong predictions** misleads the agent.

MBPO: *when* to trust the model

Imagined trips get longer during training by a **hand-made schedule**, and the model is trusted **equally everywhere**.

Contributions of the paper: (1) a guarantee of improvement inside a set $\mathcal{E}$ of accurate states; (2) a cheap uncertainty measure (GJS) to find $\mathcal{E}$; (3) adaptive imagined trips with essentially **one hyperparameter** ξ ; (4) better results than MBPO and M2AC on MuJoCo.

MACURA: *where* to trust the model

A model is accurate in some places and not in others, so every imagined trip **stops where the model itself becomes uncertain**.

Background: SAC and Probabilistic Ensembles

Soft Actor-Critic (SAC): the learner inside every method

- An **actor** (the policy) and **critics** that learn $Q(s, a)$, the expected future reward.
- **Maximum entropy:** it maximises reward plus randomness, so it keeps exploring:

$$J(\pi) = \sum_t \mathbb{E}[r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot | s_t))]$$

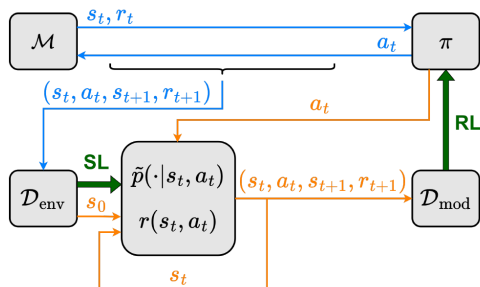
- Two critics (it takes the smaller value) and a temperature α that tunes itself.
- **Off-policy:** its target $r + \gamma Q(s', a')$ works on any stored transition, **real or imagined**.
- G = gradient updates per real step (the **update-to-data ratio**).

Probabilistic ensemble: the learned model

- $E = 7$ neural networks, each trained on its own bootstrap sample of the real data.
- Each predicts the next state as a **Gaussian**: $\tilde{p}_{\theta_e}(s' | s, a) = \mathcal{N}(\mu_{\theta_e}, \Sigma_{\theta_e})$.
- **Aleatoric** uncertainty (the world is random) $\Rightarrow$ each network predicts a **wide** Gaussian.
- **Epistemic** uncertainty (too little data) $\Rightarrow$ the networks **disagree**.
- In an imagined trip, every step uses a **random** network.

Only epistemic uncertainty means “the model may be **wrong** here”. MACURA is built on this difference.

Dyna-Style MBRL: Imagined Rollouts



Dyna-style MBRL (paper, Fig. 1).

- **Real world** (blue): the policy π acts in the real MDP $\mathcal{M}$; every transition is stored in the real buffer $\mathcal{D}_{\text{env}}$.
- **Learning** (green): the model $\tilde{p}$ learns from $\mathcal{D}_{\text{env}}$ by supervised learning (SL); SAC learns from the imagined data (RL).
- **Imagination** (orange): **branched rollouts** start from real states $s_0 \in \mathcal{D}_{\text{env}}$ and imagine a few steps with π and $\tilde{p}$; the steps go to $\mathcal{D}_{\text{mod}}$. SAC trains on 95% imagined + 5% real data.

Why it helps: much more training data per real step, and it follows the **current** policy.

The risk: every imagined step builds on the previous prediction, so errors add up, and the agent learns the model's mistakes (**model exploitation**).

⇒ The methods differ in **how far they imagine**, and **which imagined steps they keep**.

MBPO and M2AC: Two Earlier Answers

MBPO (Janner et al., 2019)

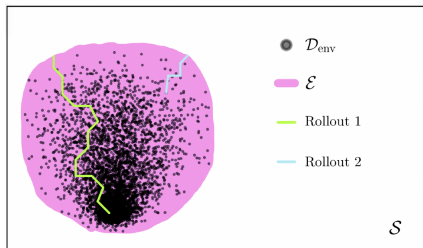
- Every imagined trip has the **same length** k . It grows during training by a **schedule** chosen by hand for each task (Humanoid $1 \rightarrow 25$, Walker 1).
- **Every** imagined step is used: the model is never checked.
- A **fixed** number G of SAC updates per real step.

⇒ It answers *when* to trust the model, not *where*.

M2AC (Pan et al., 2020): masking by uncertainty

- It measures uncertainty **one-vs-rest (OvR)**: the network that made the prediction against a Gaussian fitted to the others.
- All trips run the full horizon H . At step h it keeps only the share $\frac{H-h}{2(H+1)}$ with the **lowest** uncertainty: about 25% of all steps.
- The rule is **rank-based**: it keeps the same share when all data are bad, and throws data away when all are good.
- A **reward penalty** $r - \alpha u$ pushes the agent away from uncertain regions, which can also slow exploration.

Where to Trust Your Model?



The real data define a set $\mathcal{E}$ where the model is accurate (paper, Fig. 2).

- Inside $\mathcal{E}$, long imagined trips are useful. Once a trip **leaves** $\mathcal{E}$, its data are harmful.
- Some trips stay in $\mathcal{E}$ for a long time, others leave it at once: **one fixed length cannot fit both.**

Theorem 4.1 (simplified): if trips stop when they leave $\mathcal{E}$,

$$\eta[\pi] \geq \tilde{\eta}[\pi] - C, \quad C = 2r_{\max} \sum_t \gamma^t \sum_{\tau \leq t} \Delta p_{\mathcal{E}}[\pi]$$

η is the real return, $\tilde{\eta}$ the return in the model, and $\Delta p_{\mathcal{E}}$ the largest model error **inside** $\mathcal{E}$. If the policy improves in the model by more than C , it also improves in reality.

In practice the real error is unknown, so $\mathcal{E}$ is estimated from the model's own uncertainty:

$$\mathcal{E} = \{s \mid u(s, a) < \kappa, a \sim \pi(\cdot \mid s)\}$$

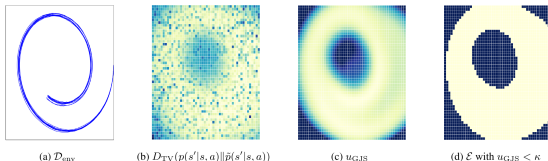
GJS Uncertainty: Do the Networks Agree?

MACURA measures the average disagreement over **all pairs** of networks ($\mathcal{N}_e$ = network e 's Gaussian):

$$u_{\text{GJS}}(s, a) = \frac{2}{E(E-1)} \sum_{e < f} D_{\text{GJS}}(\mathcal{N}_e \| \mathcal{N}_f), \quad D_{\text{GJS}} = \frac{1}{2} D_{\text{KL}}(\mathcal{N}_e \| \mathcal{N}_{ef}) + \frac{1}{2} D_{\text{KL}}(\mathcal{N}_f \| \mathcal{N}_{ef})$$

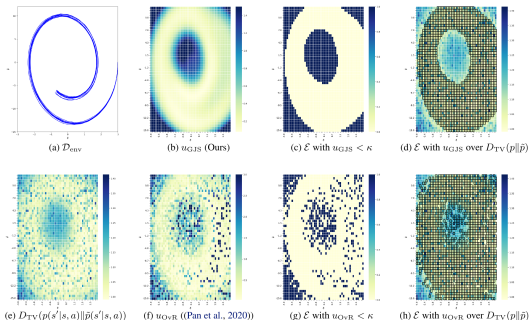
where $\mathcal{N}_{ef}$ is the **geometric mean** of the two Gaussians (again a Gaussian).

- It has a **closed form** and is symmetric, so it is cheap enough for **every** imagined step.
- It is large when two networks are **confident but predict different things**, and zero when they predict the same wide Gaussian (pure noise).



Pendulum (paper, Fig. 3): (a) the data, (b) the true model error, (c) u_{GJS} , (d) the set $\mathcal{E}$. u_{GJS} is low exactly where the real error is low.

Why a New Measure? GJS (MACURA) vs OvR (M2AC)



Top: GJS. Bottom: OvR. On the pendulum data (paper, Fig. 10).

- **GJS** (b) is smooth and follows the data. Its set $\mathcal{E}$ (c) has a clear border and matches the region of **low real error** (d).
- **OvR** (f) is **noisy** and **over-confident**: it is low in the centre, where the real error is high (e). Its set (g) misses the accurate region (h).
- The reason: OvR compares **one** network with the rest; GJS averages **all pairs**, so one lucky network cannot hide the disagreement.

⇒ To decide **where** to stop, the uncertainty must be **reliable**.

MACURA: Stop Imagining Where the Model Is Unsure

1. The stop rule

- At every imagined step, compute $u_{\text{GJS}}(s, a)$.
- If $u_{\text{GJS}} < \kappa$: keep the step and continue.
- If $u_{\text{GJS}} \geq \kappa$: **stop the trip**; this step is not kept.
- So the trip length becomes **local**: long where the networks agree, zero where they do not (at most $T_{\max} = 10$ steps).

2. A threshold that tunes itself

- Each round, take the 95% quantile (ζ) of the **first-step** uncertainties: the first step starts from a real state, so it shows what “certain” looks like now.
- $\kappa = \frac{\xi}{K} \sum_{k=1}^K \hat{u}_k$: the average over all rounds, scaled by ξ .

3. Updates that follow the trusted data

$$G = \left\lfloor G_{\max} \frac{|\mathcal{D}_{\text{mod}}|}{|\mathcal{D}_{\text{mod}}|_{\max}} \right\rfloor$$

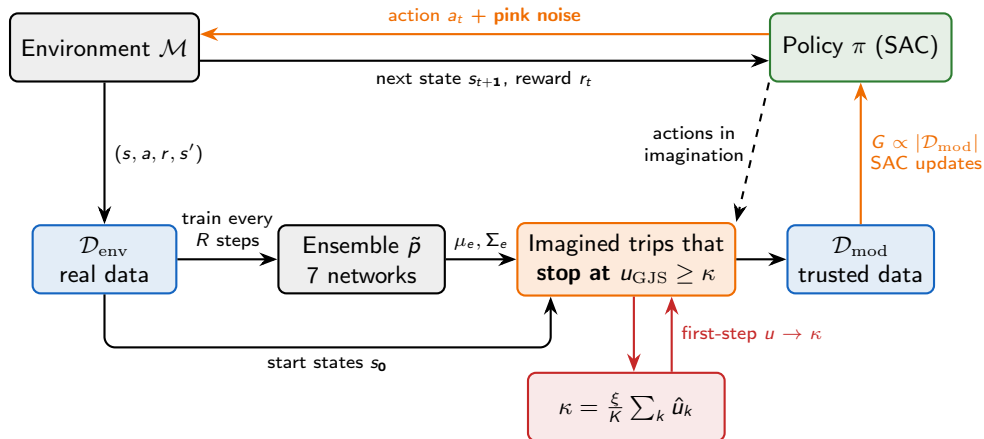
- Short trips (a poor model): few SAC updates.
- Long trips (a good model): more updates.

4. Exploration with pink noise

- Noise that is correlated in time collects new data at the border of $\mathcal{E}$, so $\mathcal{E}$ grows.

$T_{\max} = 10$ and $\zeta = 95\%$ are fixed for every task: only ξ is tuned.

MACURA Architecture



The loop is the same as MBPO's. What MACURA adds is in **orange** and **red**: trips stop where the model is uncertain, κ tunes itself, the number of updates G adapts, and the agent explores with pink noise.

How Each Method Handles Its Imagined Trips

All three start trips from real states, take a random network for each step, and train SAC on the result.

Orange: where they differ.

MBPO

```
 $k \leftarrow \text{schedule}(\text{step})$   
for  $t = 0, \dots, k - 1$  do  
   $a \sim \pi(s); s' \sim \tilde{p}_e(s, a)$   
  store  $(s, a, r, s')$   $\triangleright$  always  
   $s \leftarrow s'$   
end for  
 $G \leftarrow \text{fixed}$ 
```

mbpo.py: mbpo_rollout()

Problem: wrong steps are used as if they were right.

M2AC

```
for  $h = 0, \dots, H - 1$  do  $\triangleright$  all trips  
   $a \sim \pi(s); s' \sim \tilde{p}_e(s, a)$   
   $u \leftarrow \text{OvR}(\text{net } e \text{ vs rest})$   
  keep the  $\frac{H-h}{2(H+1)}$  lowest  $u$   
  store  $(s, a, r - \alpha u, s')$  if kept  
   $s \leftarrow s'$   $\triangleright$  nobody stops  
end for  
 $G \leftarrow \text{fixed}$ 
```

m2ac.py: m2ac_rollout()

Problem: a fixed share, whatever the quality; a noisy measure.

MACURA (paper, Alg. 2)

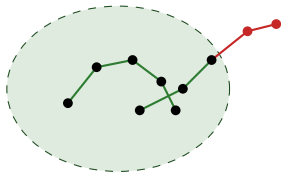
```
for  $t = 0, \dots, T_{\max} - 1$  do  
   $a \sim \pi(s); s' \sim \tilde{p}_e(s, a)$   
   $u \leftarrow u_{\text{GJS}}(s, a)$   $\triangleright$  all pairs  
  if  $t = 0$ : update  $\kappa$   
  if  $u < \kappa$  then  
    store  $(s, a, r, s'); s \leftarrow s'$   
  else  
    stop this trip  
  end if  
end for  
 $G \leftarrow G_{\max} |\mathcal{D}_{\text{mod}}| / |\mathcal{D}_{\text{mod}}|_{\max}$ 
```

macura.py: macura_rollout()

Strength: keeps exactly the steps the networks agree on.

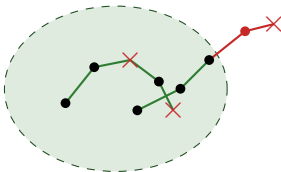
Which Imagined Data Are Kept?

MBPO



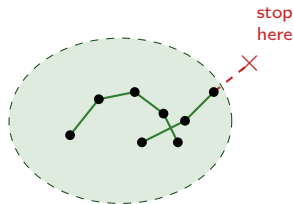
$\mathcal{E}$: model accurate
fixed length

M2AC



$\mathcal{E}$: model accurate
fixed length + masking

MACURA



$\mathcal{E}$: model accurate
stop at the border of $\mathcal{E}$

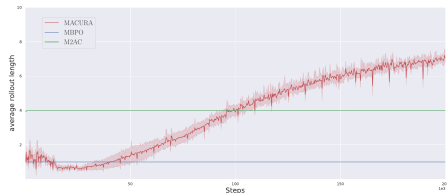
- **MBPO**: every trip has the same length, so the imagined steps **outside** $\mathcal{E}$ (red) are used for training.
- **M2AC**: the same trips, then a **fixed share** of uncertain steps is removed ($\times$), wherever they are, even inside $\mathcal{E}$.
- **MACURA**: each trip **stops at its first uncertain step**: long inside $\mathcal{E}$, nothing outside.

The Four Algorithms Side by Side

	SAC	MBPO	M2AC	MACURA
world model	none	7-network ensemble	same	same
question it answers	–	<i>when</i> to trust	<i>how much</i> to trust	<i>where</i> to trust
trip length	–	fixed k , by schedule	fixed horizon H	adaptive , until $u_{\text{GJS}} \geq \kappa$
imagined data used	–	all of it	a fixed share ($\approx 25\%$)	only steps inside $\mathcal{E}$
uncertainty	–	none	OvR (one vs rest)	GJS (all pairs)
stored reward	r	r	$r - \alpha u$	r
SAC updates per step	1	fixed G	fixed G	adaptive $G \propto \mathcal{D}_{\text{mod}} $
exploration (paper)	samples its policy	deterministic	deterministic	pink noise
tuned per task	–	the schedule	share, penalty α	only ξ

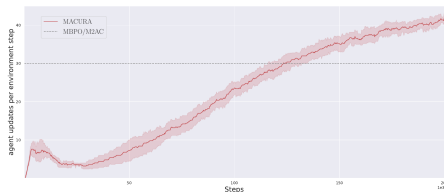
The three model-based methods share the SAC learner, the ensemble and the 95%/5% data mix. They differ **only in which imagined data reach the learner**, and how much it trains on them.

How Far Each Method Imagines, and How Much It Trains



Average trip length on Walker (paper, Fig. 17).

- **MBPO**: length 1 all along (its tuned schedule).
M2AC: a constant average.
- **MACURA**: short while the model is poor (even below 1: it can drop the first step), then growing to about 7.



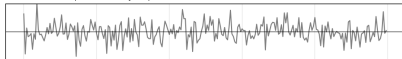
SAC updates per real step on Walker (paper, Fig. 19).

- MACURA's updates **follow its trusted data**: few at first, then more than the fixed 30 of MBPO and M2AC.
- More updates alone do not explain it: with 40 updates per step, MBPO and M2AC still stay behind.

Two Kinds of Noise

1. Exploration noise: how the agent collects data

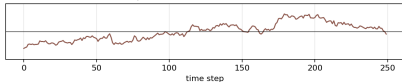
white noise: independent every step



pink noise ($1/f$): correlated in time

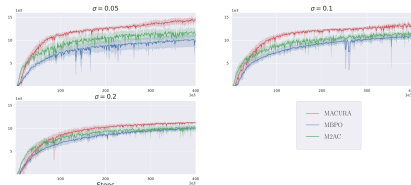


random walk (brown): drifts away



- **White noise** is independent at every step, so its pushes cancel out.
- **Pink noise** ($1/f$) is correlated in time: it pushes the same way for a while and reaches **new states**, without drifting away.

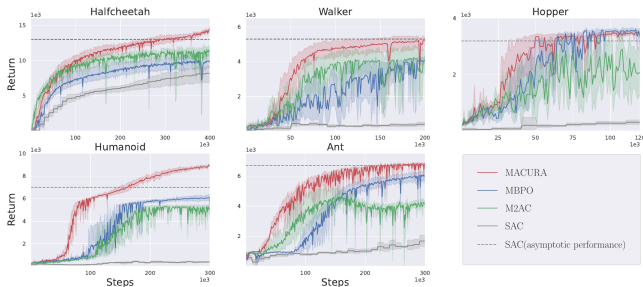
2. Environment noise: the world itself is random



HalfCheetah with hidden action noise (paper, Fig. 16).

- The networks learn noise as a **wide** Gaussian. If they agree, $u_{GJS} \approx 0$ and the step is still trusted.
- MACURA stops for **disagreement**, not for noise, and stays ahead at every noise level.

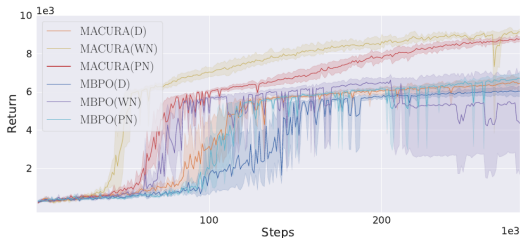
Results on MuJoCo



Return while learning on five MuJoCo tasks; mean of 5 seeds with 95% CI (paper, Fig. 4).

- MACURA learns **faster** and ends **higher** than MBPO and M2AC, most clearly on Humanoid and Ant.
- It often reaches the final return of model-free SAC with a fraction of the real data.
- The reason: long trips where the model is reliable and none where it is not, so the learner gets **more data and cleaner data**.

Exploration Changes the Results

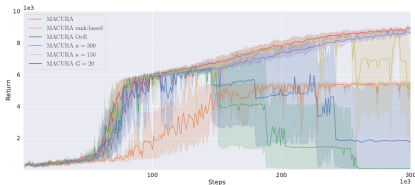


Humanoid: deterministic (D), white noise (WN) and pink noise (PN) exploration (paper, Fig. 5).

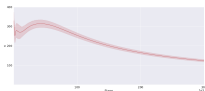
- **Deterministic**: limited but **stable**, for both methods.
- **White noise**: can be very strong for MACURA, but it **destabilises** MBPO.
- **Pink noise**: the best balance of performance and stability.
- MACURA gains the most from better exploration: new data grow $\mathcal{E}$, exactly where it uses the model.
- With deterministic exploration for **all** methods, MACURA is still the strongest on most tasks (paper appendix).

Note: the main comparison gives pink noise only to MACURA. I kept this protocol in my project.

What Matters? Ablations and the Threshold



Humanoid, one building block changed (paper, Fig. 7).



κ during training (paper, Fig. 8).

- M2AC's **rank rule** instead of a threshold: much worse.
- **OvR** instead of GJS: training **diverges**, because wrong data get through.
- A **fixed** κ : unstable at some stage. The adaptive κ starts high (rough data are enough at first) and then drops (precise data are needed later): a schedule that **emerges by itself**.
- A **fixed** number of updates: a moderate loss.
- ξ works over a **wide range**; too large exploits model errors, too small overfits.

The Paper in Short

Takeaways

- Ask *where*, not *when*, to trust the model.
- Imagined trips that stay inside $\mathcal{E}$ guarantee that improvement carries over to reality.
- GJS is a cheap, closed-form, reliable measure of **disagreement**, not of noise.
- Trips stop at the first uncertain step; κ and G tune themselves.
- One knob, ξ , and better data efficiency than MBPO and M2AC.

Limits

- ξ still differs a lot between tasks (0.3 to 30).
- Very long training destabilises MBPO and MACURA alike.
- The main comparison gives **pink noise only to MACURA**.
- Only simulated locomotion tasks.

My question: does MACURA's advantage hold on a **new task I built myself**, with the same compute for every method, and does it really trust its model less **where the model is wrong**?

Part II

My Project: MACURA Races a Drone

A new task, four algorithms, a fair comparison

The Project: Question and Hypotheses

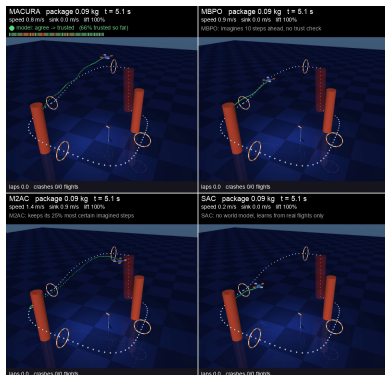
Goal: compare the four algorithms of the paper on a task where I **know** where a learned model will be wrong.

- **Why a drone:** real flights are costly and a crash breaks it, so data efficiency matters.
- **The same for all:** learner, ensemble, data mix, real steps and test scenarios. **Only the rule for imagined trips differs.**
- **MACURA** against **MBPO** (its base), **M2AC** (the closest) and **SAC** (no model).

Two hypotheses

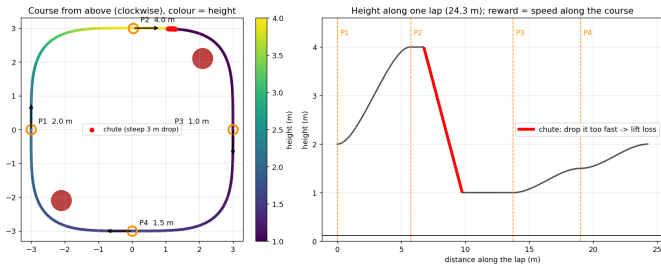
H1: with the same compute, MACURA learns faster and scores higher.

H2: MACURA trusts its model **less where the model is wrong** and trains on fewer wrong imagined data.



The four trained drones on the same new scenario.

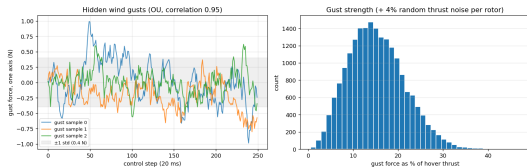
The Task: Race3



The course from above and its height profile. Red: the chute, a steep 3 m drop.

- A quadrotor in MuJoCo flies laps through **4 gates** (2, 4, 1 and 1.5 m high) around **2 pillars**.
- One lap is 24.3 m; a flight lasts **10 s** (500 steps).
- **4 actions**, 50 times per second (acceleration x and y , climb, turn), through an on-board stabiliser; **27 observations**.
- **Reward**: speed along the racing line. A crash costs 500 and ends the flight.

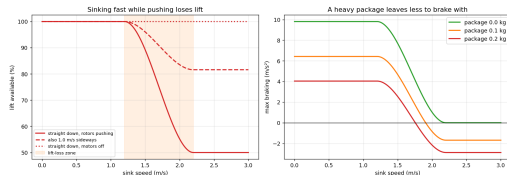
Noise and the Chute Trap



Hidden gusts: three samples over 5 s, and their strength.

Noise: MACURA should **not** stop for it

- Hidden gusts (typically 10–20% of the hover thrust) and 4% noise on each rotor.
- Visible: a steady wind and a package of random weight (0–0.2 kg).
- Exploration as in the paper: pink noise for MACURA, none for MBPO and M2AC, SAC samples its policy.



Sinking fast while pushing loses lift.

The chute: where the networks **should** disagree

- Sinking faster than **1.2 m/s** while upright and pushing loses up to **50%** of the lift, so the drone cannot brake and crashes.
- This is rare in the data and hard to predict: exactly where MACURA should **stop imagining**.

Implementation: One Loop, Three Rollout Rules

```
for step = 1, ..., 50,000 do
  a ← random (first 5,000) or explore( $\pi, s$ ) ▷ noise per method
  s', r ← env.step(a); add to  $\mathcal{D}_{\text{env}}$ 
  if model-based and step mod 250 = 0 then
    train_ensemble( $\mathcal{D}_{\text{env}}$ );  $s_0$  ← 25,000 real states
    if MACURA then
      trips ← macura_rollout( $s_0$ )
      G ← gradient_steps( $|\mathcal{D}_{\text{mod}}|$ )
    else if MBPO then
      trips ← mbpo_rollout( $s_0$ ); G ← 12
    else
      trips ← m2ac_rollout( $s_0$ ); G ← 12 ▷ M2AC
    end if
    add trips to  $\mathcal{D}_{\text{mod}}$  ▷ kept for 4 rounds
  end if
  G SAC updates on 95%  $\mathcal{D}_{\text{mod}}$  + 5%  $\mathcal{D}_{\text{env}}$  ▷ SAC: 1 on  $\mathcal{D}_{\text{env}}$ 
  every 1,000 steps: evaluate, keep the best
end for

src/training/train.py: train_one()
```

Where each part lives

- the task: envs/drone_env.py
- the ensemble: models/ensemble.py
- the three rules: algorithms/macura.py, mbpo.py, m2ac.py
- SAC: algorithms/sac.py
- every setting: config/config.py

Pipeline

- 4 Kaggle GPU notebooks on 2 accounts, each training 2 algorithms $\times$ 2 seeds in parallel (≈ 4 h).
- Download, then a testing notebook (50 new scenarios, plots, videos) and a live 3-D simulator.
- Python, PyTorch, MuJoCo, Stable-Baselines3. Runs are **deterministic** for a given seed.

A Fair Protocol, and How I Chose the Task

setting	value
real steps per run	50,000 (first 5,000 random)
imagined trips	25,000 per 250 steps, ≤ 10 steps
SAC batch	95% imagined + 5% real
MACURA	$\xi = 0.5$, $\zeta = 0.95$, $G_{\max} = 16$
MBPO	length $1 \rightarrow 10$, 12 updates/step
M2AC	$\alpha = 10^{-3}$, 12 updates/step
seeds	4, 5, 6, 7
test	50 new scenarios (3000–3049)

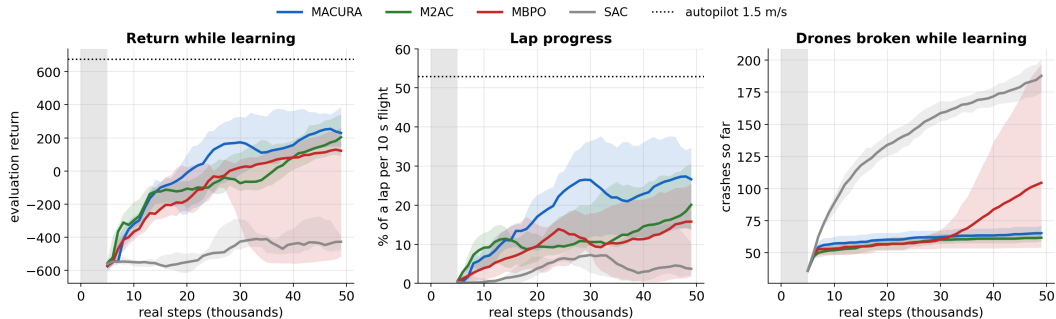
Fair by design

- **Equal compute:** 12 updates per step for MBPO and M2AC; MACURA adapts its own and averaged **10.9**.
- The best checkpoint is chosen on other scenarios (100–119): **nothing is tuned on the test**.
- IQM over seeds, plus **paired** seed-by-seed comparisons (same scenarios, wind and gusts).

How I chose the task

- Landing tasks and a first race failed; race2 was learnable, but MACURA $\approx$ MBPO.
- Race3 (a deadlier chute) was kept only if MACURA beat MBPO in at least 3 of 4 seeds (0–3): a **rule fixed in advance**. It passed.
- Then the final study on **new seeds** 4–7.

Result 1 (H1): MACURA Learns Fastest



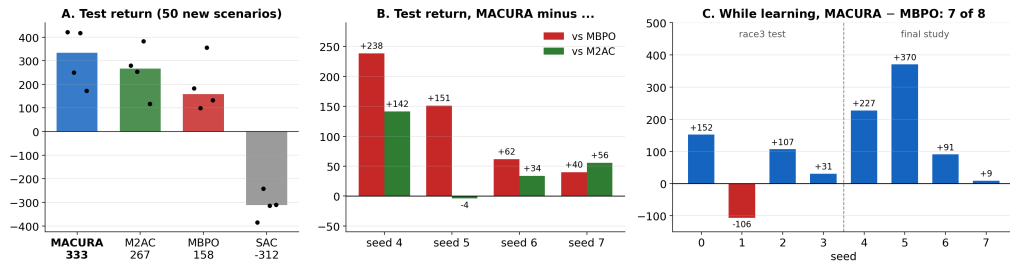
Return: MACURA is ahead from about 17k steps to the end; SAC is far behind.

Lap progress: MACURA flies the largest share of a lap (dotted: a hand-written autopilot).

Drones broken: the same for the model-based methods until MBPO's seed 5 collapses after 30k; SAC breaks by far the most.

IQM over seeds 4–7, 95% CI shaded, grey = random warm-up.

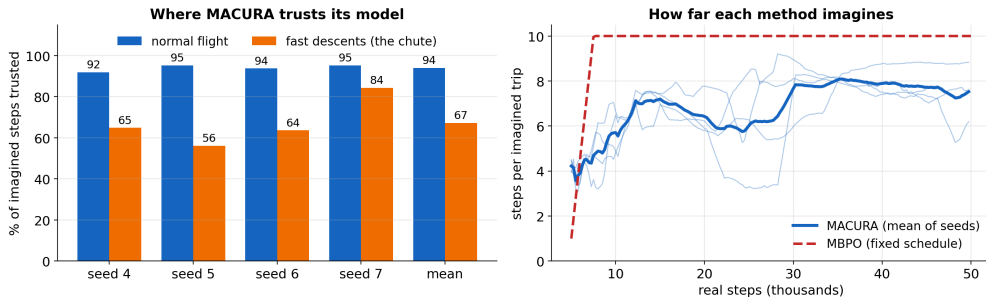
Result 2 (H1): Best on New Scenarios



	lap progress	crash rate
MACURA	31%	4%
M2AC	24%	6%
MBPO	22%	2%
SAC	7%	35%

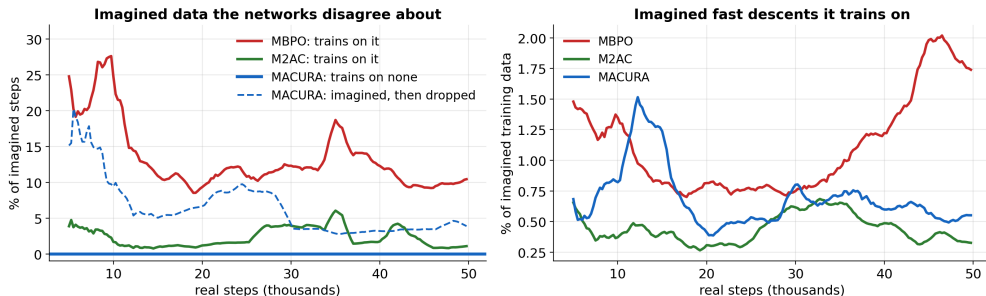
- **A:** the best test return (bars = IQM, dots = seeds).
- **B:** better than MBPO in 4 of 4 seeds, and than M2AC in 3 of 4.
- **C:** with the earlier seeds 0–3, better than MBPO in **7 of 8** ($p = 0.035$).
- MBPO crashes least, but it also flies the least far.

Result 3 (H2): Less Trust in the Chute



- **Left:** MACURA trusts **94%** of its imagined steps in normal flight but only **67%** in fast descents, in every seed. It notices where its model is unreliable.
- **Right:** its trips grow from 4 to about 8 steps as the model improves; MBPO always imagines 10.
- On real flights, the steps it would not trust have a **higher real error** (0.24 vs 0.21; with MBPO's model 0.34 vs 0.20): the signal works, but it is **noisy**.

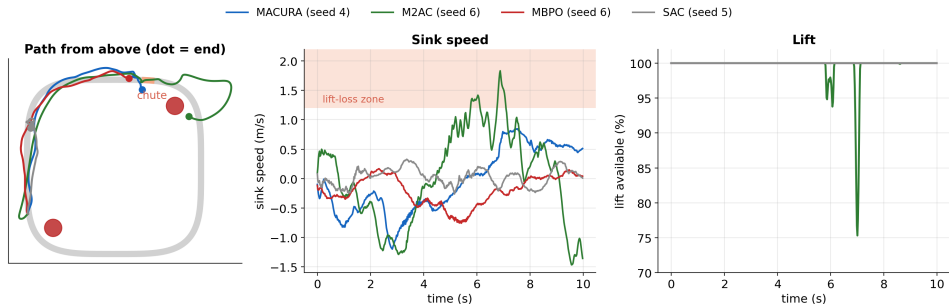
Result 4 (H2): What Each Method Trains On



- **Left:** MBPO trains on **10–25%** imagined data its own networks disagree about, M2AC on 1–6%, and **MACURA on none**: it imagines them, then drops them (dashed line).
- **Right:** late in training, MBPO trains on more and more imagined **fast descents** (up to 2%), the region where its model is least reliable.

Disagreement measured with MACURA's rule for every method; mean over seeds 4–7.

Result 5: One Test Flight, Four Drones



- **MACURA** flies furthest, reaches the chute and goes down at 0.5–0.8 m/s: its **lift stays at 100%**.
- **M2AC** dives at up to **1.8 m/s**, enters the lift-loss zone and **loses up to 25%** of its lift: the trap in action.
- **MBPO** is slower and only just reaches the top; **SAC** barely leaves the start.
- This is one example (scenario 3000, best seed of each), not a statistic: in the 50-scenario test the drones rarely sink that fast.

Limitations

Training budget and statistics

- Only **50,000 real steps** per run, against 125k–400k in the paper. The curves are still rising, so I compare **early learning**, not final performance.
- The task is **not solved**: 31% of a lap per 10 s, while a simple autopilot reaches 54%.
- Kaggle's limits (12 h per session, about 30 GPU hours per week) did not allow longer runs.
- **4 seeds** per algorithm (8 for MACURA vs MBPO): a consistent result, but only at the edge of significance, and no hyperparameter search.

Fairness and the task

- Only MACURA uses **pink noise** (the paper's protocol), so part of its gain may come from exploration.
- MBPO's schedule was not tuned for my task.
- **One task**, designed and changed by me five times. The rule fixed in advance and the new seeds reduce this risk.
- The drones **rarely reach the chute trap**, and the disagreement predicts the real error only weakly.
- **Simulation only.**

Conclusion and Next Steps

The paper

- Trust the model only **where it trusts itself**.
- Imagined trips stop at the first uncertain step ($GJS \geq \kappa$).
- κ and G tune themselves; one knob, ξ .
- GJS reacts to disagreement, not to noise.

My project

- **H1 supported**: the best test return (333 vs 267, 158 and -312), better than MBPO in **7 of 8** seeds, and no collapse.
- **H2 partly**: less trust in fast descents (67% vs 94%) and no training on data the networks disagree about; but the signal is noisy and the chute is rarely reached.
- With 50k steps: **consistent early-learning evidence, not yet proof**.

Next steps

- Train longer (100k–200k steps), so the chute matters.
- 10 or more seeds.
- The same exploration for all methods.
- Tune ξ .
- Fly the best policies on a real drone.

Thank you for your attention!

Code, results and videos: github.com/silvergjecha22/macura-drone